QML Lab • research project 07

Quantum GAN for a One-Dimensional Distribution

Learning and loading a probability distribution with a quantum generator.

Living document • last updated October 25, 2026

Quadrant	CQ (classical data, quantum processor)	**Track**	Variational
Difficulty	Hard	**Tools**	Qiskit Machine Learning, PyTorch
Builds on	Modules 5 and 7 of the QML Lab	**Workspace**	`qml_projects/quantum-gan-1d/`

Summary. A quantum generative adversarial network (qGAN) trains a parameterised circuit whose measurement outcomes follow a target distribution, guided by a classical discriminator. Besides generating samples, the trained circuit loads the distribution into amplitudes, which other quantum algorithms need as input. The project studies how well a qGAN learns discretised one-dimensional distributions, how stable the training is, and how it compares with small classical generators.

1 Background and key concepts

1.1 Generative adversarial networks

A GAN (Goodfellow et al., 2014) trains a generator G and a discriminator D in a two-player game,

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{\text{data}}}\big[\log D(x)\big] + \mathbb{E}_z\big[\log\big(1 - D(G(z))\big)\big]. \tag{1}$$

The generator improves until the discriminator can no longer tell its samples from real data.

1.2 Quantum generators

Quantum versions were proposed by Lloyd and Weedbrook (2018) and Dallaire-Demers and Killoran (2018). In the qGAN of Zoufal et al. (2019) the generator is an n-qubit circuit $U(\theta)$ whose output state

$$|g_\theta\rangle = U(\theta)\,|0\rangle^{\otimes n} = \sum_{j=0}^{2^n-1} \sqrt{p_\theta(j)}\, e^{i\varphi_j}\,|j\rangle \tag{2}$$

defines a distribution over 2^n bins: measuring returns bin j with probability $p_\theta(j)$. The discriminator is a classical neural network that sees the sampled bin values. After training, the circuit is an efficient loader for the learned distribution.

1.3 Measuring success

Quality is measured by the relative entropy (Kullback–Leibler divergence) to the target q,

$$D_{\text{KL}}(q \,\|\, p_\theta) = \sum_j q(j) \log \frac{q(j)}{p_\theta(j)}, \tag{3}$$

and by the total variation distance. Training curves often oscillate, as in classical GANs.

2 Research questions

The project is open-ended: these questions are the starting point, not the finish line. Add new ones to the README and the journal as they come up.

RQ1 How many qubits (bins) are needed for a useful approximation of log-normal and bimodal targets?

RQ2 Is training stable, or does it oscillate or collapse to one mode?

RQ3 How does the KL divergence compare with a small classical GAN and with a directly parameterised histogram?

3 Method

3.1 Data

Targets discretised to 2^n bins: a log-normal distribution (as in Zoufal et al., 2019), a truncated normal and a bimodal mixture. Training samples are drawn from the discretised targets.

3.2 Models

Generator: hardware-efficient circuit with R_Y rotations and CZ entanglers, L layers, initialised from a uniform or normal-like distribution. Discriminator: small PyTorch MLP (Paszke et al., 2019). Alternating updates with Adam; generator gradients through Qiskit Machine Learning's PyTorch connector.

3.3 Baselines

A classical GAN with a small MLP generator; a softmax-parameterised histogram with $2^n - 1$ parameters fitted by maximum likelihood (the best possible fit for the bins).

3.4 Metrics

- KL divergence and total variation distance to the target over training.
- Oscillation amplitude of the loss curves (training stability).
- Number of parameters and circuit evaluations.

4 Planned experiments

Each experiment becomes one `[[experiments]]` entry in the workspace, so its runs, parameters and metrics are stored and compared in the portal.

5 Running it in the QML Lab

The workspace lives in `qml_projects/quantum-gan-1d/`. Scripts import three helpers from `qml_lab/plots.py`:

ID	Experiment	What varies	Main metrics
E1	Log-normal, 3 qubits	epochs	KL divergence
E2	Resolution	$n = 3, 4, 5$ qubits	KL, cost
E3	Bimodal target	mixture weights	mode coverage
E4	Generator depth	$L = 1..4$	KL, stability
E5	Shots per step	$M \in \{100, 1000\}$	KL, stability

Table 1: Experiment plan. Results go into the journal and the results log (section 9).

- `params()` reads the parameters from the portal's run form (defaults when run by hand);
- `save()` stores figures with the run;
- `log_metrics()` records numbers that appear in the run table.

```
qml_projects/quantum-gan-1d/
  project.toml          metadata + experiments
  README.md             goal, open questions, ideas
  experiments/          runnable scripts
  docs/main.tex         this document
```

First experiment: This experiment is *planned*: write the script, add the block below to `project.toml`, then run `python manage.py sync_projects`.

```
[[experiments]]
slug = "lognormal"
title = "qGAN for a 3-qubit log-normal distribution"
script = "experiments/e01_lognormal.py"
figure = "distribution.png"

[experiments.params]
n_qubits = 3
n_layers = 2
epochs = 300
shots = 1000
target = "lognormal"
seed = 0
```

```
# from the portal folder: pick up changes to project.toml
python manage.py sync_projects
cd qml_projects/quantum-gan-1d
# run by hand with the default parameters
PYTHONPATH=../../qml_lab python experiments/e01_lognormal.py
```

6 Quality bar

- A classical baseline with a similar number of trainable parameters, tuned with the same care (Bowles et al., 2024).
- At least five random seeds per configuration, reported as mean $\pm$ standard deviation.
- How results change with the number of qubits.
- At least one run under a realistic noise model.

- Show the learned histogram next to the target and the classical fits in every report.

7 Open directions

- Use the trained loader as input to amplitude estimation, the original application of Zoufal et al. (2019).
- Quantum circuit Born machines trained with maximum mean discrepancy instead of a discriminator.
- Two-dimensional distributions on more qubits.

8 Risks and limitations

- GAN training is unstable; use several seeds and report the spread.
- For 8 bins a classical histogram fits exactly; the interest is in loading and scaling, not in beating it.

9 Results log

Summarise findings here as they become stable; day-to-day notes belong in the portal journal.

Date	Run	Finding	Next step

References

Bowles, J., Ahmed, S. and Schuld, M. (2024) 'Better than classical? The subtle art of benchmarking quantum machine learning models', Preprint, arXiv:2403.07059. Available at: https://arxiv.org/abs/2403.07059.

Dallaire-Demers, P.-L. and Killoran, N. (2018) 'Quantum generative adversarial networks', *Physical Review A*, 98(1), 012324. Available at: https://doi.org/10.1103/PhysRevA.98.012324.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A. and Bengio, Y. (2014) 'Generative adversarial nets', in *Advances in Neural Information Processing Systems 27 (NIPS 2014)*, pp. 2672–2680.

Lloyd, S. and Weedbrook, C. (2018) 'Quantum generative adversarial learning', *Physical Review Letters*, 121(4), 040502. Available at: https://doi.org/10.1103/PhysRevLett.121.040502.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J. et al. (2019) 'PyTorch: An imperative style, high-performance deep learning library', in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, pp. 8024–8035.

Zoufal, C., Lucchi, A. and Woerner, S. (2019) 'Quantum generative adversarial networks for learning and loading random distributions', *npj Quantum Information*, 5, 103. Available at: https://doi.org/10.1038/s41534-019-0223-2.